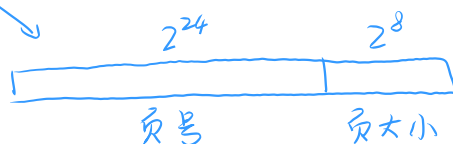


2.2 (32分) 假设一个系统的页面大小为 256 字节，给某个进程分配了 7 个帧：

序号	物理地址 (十六进制)
0	0600
1	0400
2	1C00
3	0C00
4	1200
5	1600
6	1800



最近最少使用

采用 LRU 页面置换算法，该进程按照如下两种方式运行：

```
int a[][] = new int[256][64];
for(int j=0; j < 64; j++)
    for(int i = 0; i < 256; i++)
        a[i][j] = 0;
```

(A)

先列后行

```
int a[][] = new int[256][64];
for(int i=0; i < 256; i++)
    for(int j = 0; j < 64; j++)
        a[i][j] = 0;
```

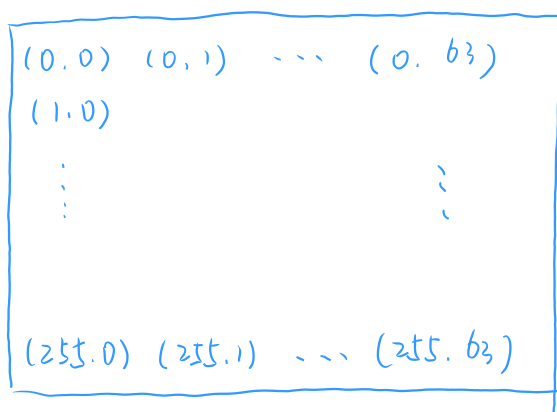
(B)

先行后列

假设该进程的地址空间大小也为 7 个页面，进程代码占用了页面 0 并已经映射到帧 0，且常驻物理内存，其余 6 页面用来保存数组 a 的数据，但尚未映射到物理内存。临时变量 i 和 j 保存在寄存器中，一个 “int” 占用 4 个字节。

- (1) 计算 A 和 B 各自从 $i=0, j=0$ 运行到 $i=28, j=7$ 时发生了多少次缺页异常。(8分)
- (2) 对于 A 和 B，分别计算 $i=28, j=7$ 时帧 1-6 所存储的数组 a 的行号。(12分)
- (3) 对于 A 和 B，分别计算 $i=28, j=7$ 时所有在物理内存中数组 a 的行首元素的物理地址 (用十六进制表示)。(12分)

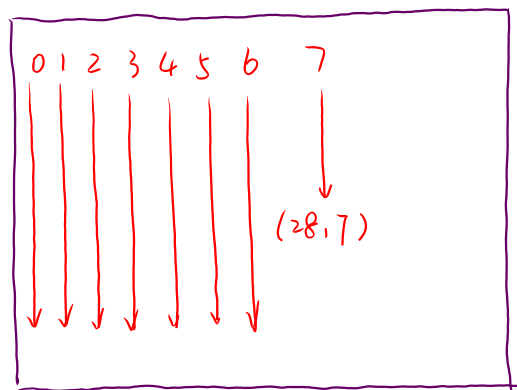
数组 a



每行是 $64 \times 4 = 256$ B
刚好是一页，因编译器是按行进行一维化操作，因此想要不发生 page fault，一共需要 256 帧物理内存

(1)

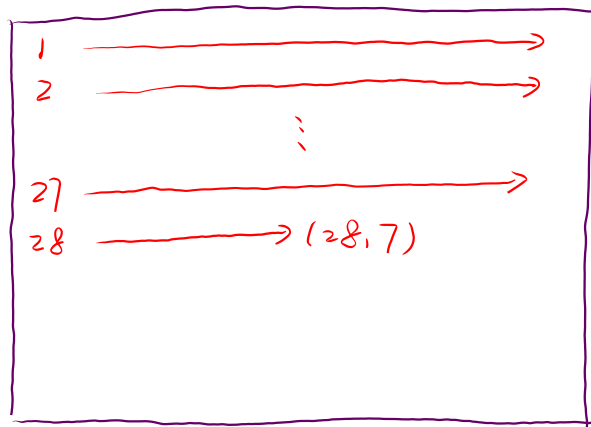
A 先列后行



此时每访问一个元素行号都会改变，即每个元素都会产生 page fault。一共产生：

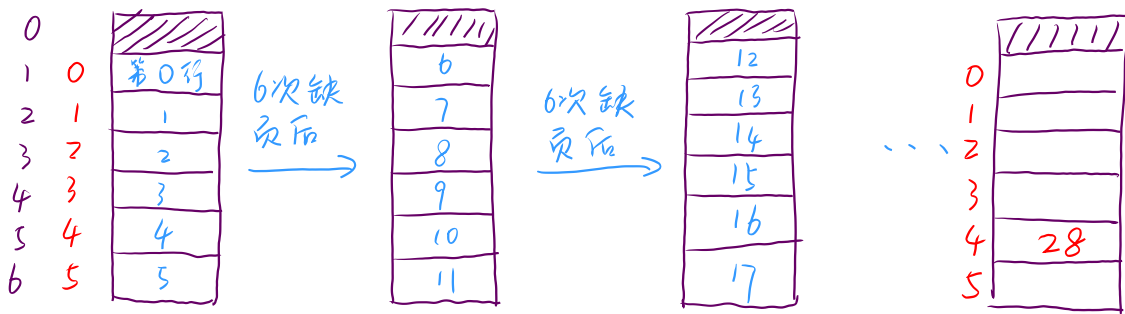
$$7 \times 256 + 29 = 1821 \text{ (次)}$$

B 先行后到



此时每一行即一页,存放在一个物理帧中,只会产生一次 page fault。
因此 - 共产生 29 次

(2). 每发生 page fault 就将替换掉一个物理帧
先定位到行号 28 所在的物理帧。



规律: $(\text{page fault 次数} - 1) \% 6$

∴ A 先到后行:

$$(1821 - 1) \% 6 = 303 \dots 2$$

1		
2	0	26
3	1	27
4	②	28
5	3	23
6	4	24
7	5	25

B 先行后到:

$$(29 - 1) \% 6 = 4 \dots 4$$

1		
2	0	24
3	1	25
4	2	26
5	3	27
6	④	28
7	5	23

(3) 由 (2) 得 且行首元素位于每一帧的起始位置, 帧set=0
前32位是帧号 后8位

A

	0600		
26	0400	→ 26行首元素地址	0400
27	1C00	27	1C00
28	0C00	28	0C00
23	1200	23	1200
24	1600	24	1600
25	1800	25	1800

B

	0400	→ 24行首元素地址	0400
24	1C00	25	1C00
25	0C00	26	0C00
26	1200	27	1200
27	1600	28	1600
28	1800	23	1800

8页
1、假设一个系统的页面大小为 2^{11} 字节, 给某进程分配了 8 个物理帧:

帧号	物理地址 (二进制)
0	00101 000000000000
1	00001 000000000000
2	11011 000000000000
3	11010 000000000000
4	10001 000000000000
5	10101 000000000000
6	11000 000000000000
7	00101 000000000000

采用 LRU 页面置换算法, 如该进程按照如下两种方式运行:

```
int a[][] = new int[512][512];
for(int j = 0; j < 512; j++)
    for(int i = 0; i < 512; i++)
        a[i][j] = 0;
```

(A)

```
int a[][] = new int[512][512];
for(int i = 0; i < 512; i++)
    for(int j = 0; j < 512; j++)
        a[i][j] = 0;
```

(B)

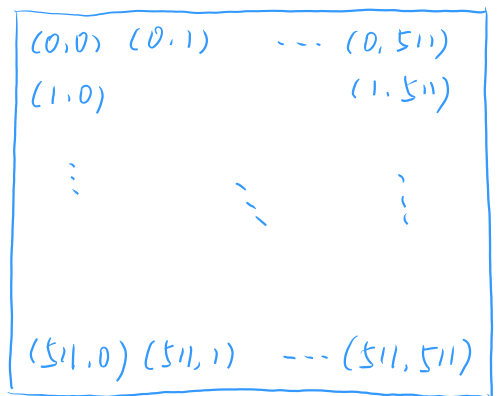
假设进程的地址空间大小也为 8 个页面, 进程代码占用了页面 0 并已经映射到帧号 0, 且常驻物理内存。其余 7 页面用来保存数组 a 的数据, 但尚未映射到物理内存。临时变量 i 和 j 保存在寄存器中, 一个 "int" 占用 4 个字节。

(1) 假设内存访问时间为 $3\mu s$, 缺页异常处理时间为 $1ms$ 。计算 A 和 B 各自从 $i=0, j=0$ 运行到 $i=8, j=6$ 时所花的时间, 需列计算公式。(18 分)

(2) 对于 A 和 B, 分别计算 $i=8, j=6$ 时 1-7 帧物理内存所存储的数组 a 的行号 (即 i 值)。(7 分)

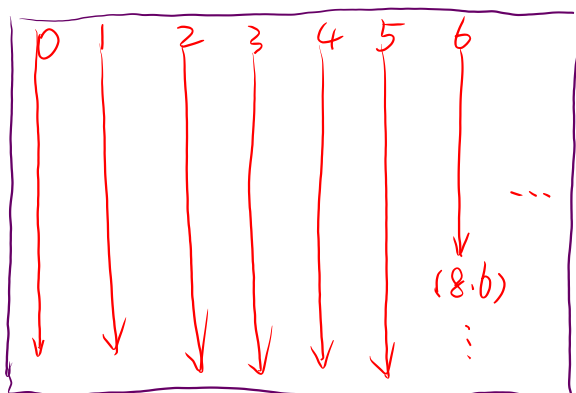
(3) 对于 A 和 B, 分别计算 $i=8, j=6$ 时所有在物理内存中数组 a 的行首元素的物理地址和逻辑地址 (用二进制表示)。(15 分)

数组 a



每行大小是 $512 \times 4 = 1024 \text{ B}$
 刚好是一页大小, 因此
 想不发生缺页错误,
 一共需要 512 帧物理
 内存。

(1). A 先到后行



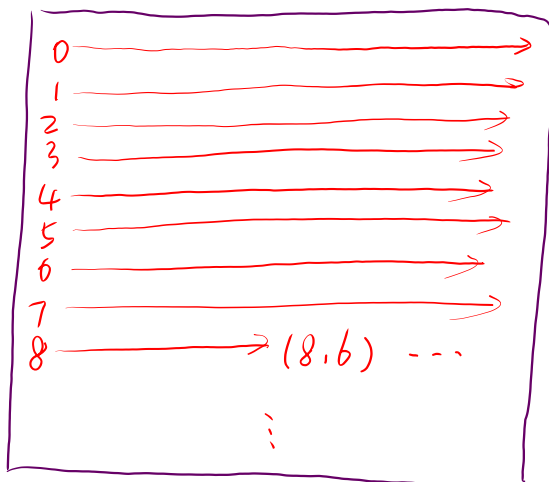
每访问一个元素行号都会
 改变, 即每个元素都会产
 生缺页错误。一共产生:

$$6 \times 512 + 9 = 3081 \text{ 次}$$

$$t_1 = 3081 \times 1000 + (6 \times 512 + 9) \times 3$$

$$= 3090243 \text{ us} \quad \text{访问的个数}$$

B. 先行后列



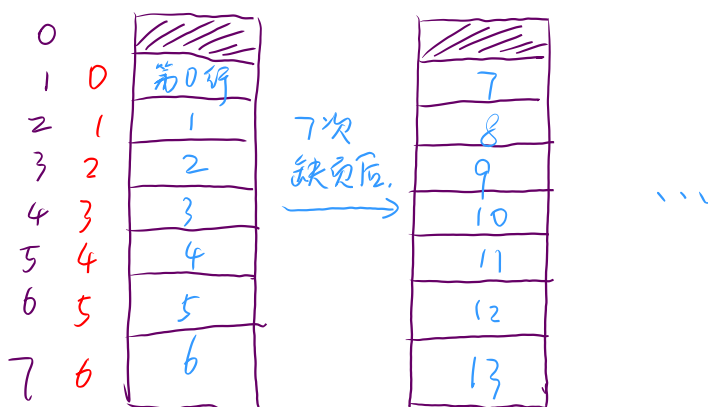
此时一行即一页, 存放
 在一个物理帧中, 因此
 访问同一行元素只会产
 生一次缺页错误, 一共
 产生: 9 次。

$$t_2 = 9 \times 1000 + (8 \times 512 + 7) \times 3$$

$$= 21309 \text{ us} \quad \text{访问的个数}$$

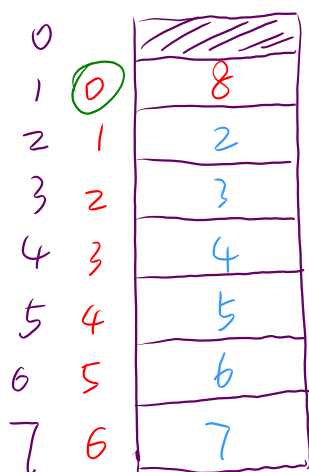
(2) 每次发生缺页错误都会替换一个物理帧

规律: $(\text{缺页错误次数} - 1) \% 7$ 先定位行号8的位置.



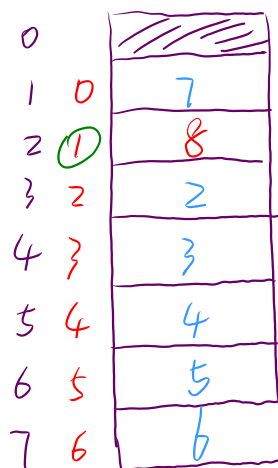
A 先到后行:

$$(3081 - 1) \% 7 = 440 \dots 0$$



B 先行后列:

$$(9 - 1) \% 7 = 1 \dots 1$$



(3). A 先行后行

帧号	行号	首元素物理地址	首元素逻辑地址
0	—		
1	8	0000 000000000000	同左
2	2	11011 同上	“ ”
3	3	11010 “ ”	“ ”
4	4	10001 “ ”	“ ”
5	5	10101 “ ”	“ ”
6	6	11000 “ ”	“ ”
7	7	00101 “ ”	“ ”

B 先行后列

帧号	行号	首元素物理地址	首元素逻辑地址
0	—		
1	7	0000 000000000000	同左
2	8	11011 “ ”	“ ”
3	2	11010 “ ”	“ ”
4	3	10001 “ ”	“ ”
5	4	10101 “ ”	“ ”
6	5	11000 “ ”	“ ”
7	6	00101 “ ”	“ ”

2K
3、(6分) 假设有一个虚拟内存系统的页面大小为 2048 字节，采用 LRU 页面置换算法，每一个进程分配 3 个页帧的物理内存。现有两个进程分别运行如下的代码：

```
int a[][] = new int[512][512];
for(int j = 0; j < 512; j++)
    for(int i = 0; i < 512; i++)
        a[i][j] = 0;
```

(A)

```
int a[][] = new int[512][512];
for(int i = 0; i < 512; i++)
    for(int j = 0; j < 512; j++)
        a[i][j] = 0;
```

(B)

$512 \times 4 = 1024 \text{ B}$
一行即一页。

如果代码用了 1 个页帧并常驻内存，其余 2 帧用来保存数组 a 的数据，临时变量 i 和 j 保存在寄存器中，一个 “int” 占用 4 个字节。请计算 A 和 B 各自产生了多少次缺页异常。(必须写出计算过程)

A: 先列后行遍历数组 a:

每个元素都产生缺页异常: $512 \times 512 = 262144$ 次

B: 先行后列遍历:

同一行元素只会产生一次缺页异常: 512 次

8.13 考虑下面的一个系统在某一时刻的状态：

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	<u>A B C D</u>	<u>A B C D</u>	<u>A B C D</u>
P_0	0 0 1 2	0 0 1 2	1 5 2 0
P_1	1 0 0 0	1 7 5 0	
P_2	1 3 5 4	2 3 5 6	
P_3	0 6 3 2	0 6 5 2	
P_4	0 0 1 4	0 6 5 6	

使用银行家算法回答下面问题：

- $Need$ 矩阵的内容是怎样的？
- 系统是否处于安全状态？
- 如果从进程 P_1 发来一个请求 $(0, 4, 2, 0)$ ，这个请求能否立刻被满足？